

UNIVERSIDAD TECNOLÓGICA DEL PERÚ

Carrera de Ingeniería de Software

SEGUNDO AVANCE DEL PROYECTO FINAL

Pruebas unitarias, de integración, funcionales, de rendimiento y de seguridad básica para validar funcionalidades del sistema de gestión veterinaria Spring Petclinic

Curso:
Pruebas de Software

Docente:
Ing. Mgtr. Jonathan Merino Farias

Integrantes:
Marcelo Fabrizio Araujo Rosas
Alvaro Christian Pilco Gallardo
Helam Rojas Ramón

Lima – Perú
2026

Índice

1. Introducción
2. Planteamiento del problema
3. Justificación
4. Objetivos
5. Sistema, alcance y limitaciones
6. Marco teórico y metodología
7. Requisitos y trazabilidad
8. Plan de pruebas
9. Diseño de casos de prueba
10. Pruebas unitarias e integración
11. Pruebas funcionales
12. Pruebas básicas de carga local
13. Pruebas de seguridad básica
14. Pruebas móviles, de usabilidad y accesibilidad (hito posterior)
15. Pruebas de regresión (hito posterior)
16. Registro de incidencias y correcciones
17. Métricas y análisis de resultados
18. Conclusiones (hito posterior)
19. Referencias
20. Anexos

1. Introducción

El proyecto evalúa Spring Petclinic, una aplicación web desarrollada en Java con Spring Boot para gestionar propietarios, mascotas y visitas de una clínica veterinaria. Se mantienen las seis funcionalidades del primer avance: registrar, buscar y actualizar propietarios; registrar y actualizar mascotas; y registrar visitas.

Este segundo avance actualiza las secciones 1 a 10 y documenta la ejecución de los casos CP01 a CP06, el análisis y la corrección de las dos fallas iniciales, las pruebas funcionales, una prueba básica de carga local y pruebas de seguridad básica sobre una instancia local. También registra las incidencias encontradas y las métricas obtenidas.

La ejecución consolidada se realizó el 29 de octubre de 2026 sobre el commit 500158f del repositorio oficial, con MySQL 8.0.46 como base de datos principal y H2 como base de comparación. El 30 de octubre de 2026 se revisó el avance: se reclasificó S08 de aprobado a fallido sin nueva ejecución, se ejecutó una tercera corrida de los casos funcionales que verifica también los mensajes mostrados, se completó la documentación de cada caso y se precisó la clasificación de E01, de las pruebas de carga y de las pruebas de seguridad. Los resultados se refieren a los escenarios ejecutados y no acreditan la ausencia de defectos en el resto del sistema.

2. Planteamiento del problema

2.1. Descripción del problema

La gestión de propietarios, mascotas y visitas requiere datos válidos y asociaciones consistentes. Un registro incompleto o una relación incorrecta puede afectar la consulta y actualización de la información. Se evalúa si las funcionalidades seleccionadas cumplen los requisitos definidos, sin asumir previamente la existencia de defectos.

Los antecedentes citados abordan la evaluación de sistemas de información en salud (Noël et al., 2022), la calidad de los artefactos de prueba (Tran et al., 2021), las pruebas de integración (Shin et al., 2013) y las prácticas de regresión (Minhas et al., 2020). Sus contextos difieren del presente proyecto; se emplean para organizar la evaluación, no como evidencia de fallos en Spring Petclinic.

2.2. Formulación del problema

¿En qué medida las funcionalidades seleccionadas de Spring Petclinic cumplen los requisitos de validación, almacenamiento y asociación de datos al evaluarse mediante pruebas unitarias, de integración, funcionales, de rendimiento básico y de seguridad básica?

3. Justificación

3.1. Justificación técnica

Las pruebas permiten contrastar el comportamiento de las funcionalidades con criterios explícitos. La evaluación distingue las validaciones de componentes individuales, las operaciones que requieren la base de datos y los flujos completos accesibles por HTTP. Esta separación facilita identificar el alcance de cada caso y la causa de cada diferencia.

3.2. Justificación académica

El proyecto aplica el diseño, la ejecución y la documentación de pruebas sobre una aplicación existente. Relaciona requisitos con casos, evidencias e incidencias, y distingue entre un defecto del producto, un problema del instrumento de prueba y un problema del ambiente.

3.3. Justificación práctica

Los scripts, reportes y la matriz editable permiten repetir las verificaciones y comprobar las correcciones. El trabajo se delimita a las funcionalidades seleccionadas para mantener una evaluación viable con los recursos y el periodo académico disponibles.

4. Objetivos

4.1. Objetivo general

Evaluar el cumplimiento de los requisitos de las funcionalidades seleccionadas de Spring Petclinic mediante pruebas unitarias, de integración, funcionales, de rendimiento básico y de seguridad básica.

4.2. Objetivos específicos

- Diseñar y ampliar casos de prueba vinculados con los requisitos de propietarios, mascotas y visitas.
- Ejecutar pruebas unitarias y de integración, analizar sus fallas y comprobar las correcciones.
- Ejecutar pruebas funcionales con datos válidos, inválidos y valores límite sobre los seis flujos.
- Medir tiempos de respuesta y errores bajo una carga local acotada.
- Verificar controles de seguridad pertinentes: autenticación existente, validación de entradas, asociación propietario-mascota y exposición de información.
- Registrar incidencias y métricas con evidencias verificables.

5. Sistema, alcance y limitaciones

5.1. Descripción del sistema

Spring Petclinic se usa como sistema existente bajo prueba, sin modificar su código de producción. Las funcionalidades seleccionadas se agrupan en dos módulos.

Módulo funcional	Funcionalidades seleccionadas
Gestión de propietarios	Registro, búsqueda por apellido y actualización de datos.
Gestión de mascotas y visitas	Registro y actualización de mascotas asociadas a un propietario; registro de visitas para una mascota.

5.2. Reglas de negocio y validaciones

Antes de ejecutar se contrastó cada regla con el código del commit evaluado. La columna de verificación indica dónde se implementa; el resultado de las pruebas se presenta en las secciones 10 a 13.

Código	Regla	Implementación verificada en el código
RN01	Dirección, ciudad y teléfono del propietario obligatorios; teléfono de exactamente diez dígitos.	Owner: @NotBlank y @Pattern(\d{10})
RN02	Toda mascota debe asociarse a un propietario existente.	Relación Owner–Pet (@OneToMany, owner_id)
RN03	La mascota requiere nombre, tipo y fecha de nacimiento.	PetValidator
RN04	Sin nombres de mascota duplicados para un mismo propietario.	PetController y restricción única (owner_id, name)
RN05	La fecha de nacimiento no puede ser posterior a la fecha actual.	PetController (alta y edición)
RN06	La visita se asocia a una mascota del propietario indicado y tiene descripción no vacía.	VisitController y Visit: @NotBlank
RN07	La fecha de una nueva visita debe ser posterior al día actual.	VisitController: rechaza hoy y fechas pasadas

5.3. Alcance del segundo avance

El avance comprende la ejecución de CP01 a CP06 y su corrección, 31 casos funcionales (F01–F31), 13 verificaciones de seguridad básica (S01–S13), una prueba adicional de concurrencia (E01), dos escenarios básicos de carga local y el registro de incidencias y métricas. Las secciones 14, 15 y 18 corresponden a hitos posteriores.

5.4. Limitaciones del proyecto

- **Ambiente.** Las ejecuciones se realizaron en un contenedor Linux local (2 vCPU, 7,8 GiB). La ejecución de transferencia usó Windows con Temurin 21.0.13+9; la consolidada usó OpenJDK 21.0.12.1. Ambas cumplen el requisito java.version=17 del proyecto.
- **Base de datos.** El plan preveía MySQL. Se instaló MySQL 8.0.46 y se ejecutaron sobre él las pruebas de integración, funcionales, de seguridad y de carga. H2 se usó como comparación; sus resultados se identifican por separado.
- **Seguridad.** La variante evaluada no implementa autenticación ni autorización. Las pruebas son básicas, se limitan a la instancia local y no constituyen una prueba de penetración ni una auditoría.

- **Carga.** Cada escenario se ejecutó una vez y el generador de carga compartió el equipo con la aplicación. Los tiempos no representan la capacidad de un entorno de producción ni el límite del sistema.
- **Pruebas manuales.** Los flujos de interfaz se recorrieron con un navegador automatizado. No se registraron pruebas manuales realizadas por integrantes del equipo.
- **Responsables.** La asignación nominal de responsables por caso no consta en las fuentes y queda pendiente de registro por el equipo.

6. Marco teórico y metodología

6.1. Conceptos fundamentales

Pruebas de software. Actividades que contrastan el comportamiento de un sistema con criterios definidos. Sus resultados aportan evidencia sobre los escenarios examinados; no demuestran la ausencia de defectos.

Pruebas unitarias. Comprueban una unidad de comportamiento de forma aislada. Aquí se aplican a las validaciones del propietario y de la mascota.

Pruebas de integración. Comprueban la interacción entre componentes, en este caso repositorios JPA, base de datos y controladores (Shin et al., 2013).

Pruebas funcionales. Verifican qué hace el sistema frente a entradas y condiciones, a partir de su comportamiento observable (Universidad Tecnológica del Perú [UTP], 2026b). Aquí se automatizaron con solicitudes HTTP reales y con un navegador automatizado.

Técnicas de diseño. La partición de equivalencia divide los datos en clases que deberían comportarse igual y prueba un representante de cada clase; el análisis de valores límite prueba los extremos, justo debajo y justo encima (UTP, 2026a). Los casos se clasifican además como positivos, cuando confirman que la función opera con datos válidos, o negativos, cuando comprueban que el sistema rechaza entradas inválidas de forma controlada.

Pruebas exploratorias y ad hoc. Una sesión exploratoria integra aprendizaje, diseño y ejecución bajo una misión (charter), un tiempo delimitado (timebox), heurísticas, registro de evidencias y un resumen final (debrief). Una prueba ad hoc es una investigación informal, con menor estructura, que también debe registrar sus hallazgos (UTP, 2026b).

Pruebas de rendimiento. Evalúan la respuesta del sistema bajo condiciones de trabajo definidas. Una prueba de carga verifica la demanda prevista; una prueba de estrés supera esa demanda para encontrar el punto de degradación y observar la recuperación (UTP, 2026c).

Pruebas de seguridad. Verifican si la aplicación protege información y funciones. Se distinguen el análisis de vulnerabilidades, la prueba de penetración y la auditoría (UTP, 2026e). Este avance realiza un análisis básico sobre la aplicación en ejecución, con referencia a categorías del OWASP Top 10 (OWASP Foundation, 2021).

Caso de prueba y trazabilidad. El caso define identificador, objetivo, precondiciones, datos, pasos y resultado esperado; tras ejecutarlo se registran obtenido, estado y evidencia (Tran et al., 2021; UTP, 2026a). La trazabilidad vincula requisitos, casos, resultados e incidencias.

Incidencia. Diferencia entre esperado y obtenido. Se analiza considerando requisitos, datos y ambiente antes de clasificarla como defecto del producto, problema del instrumento de prueba o problema del ambiente.

6.2. Metodología de trabajo

Se mantiene la adaptación de Scrum con seis sprints de dos semanas entre las semanas 6 y 17. Una tarea de ejecución se considera terminada cuando cuenta con resultado y evidencia; el diseño de un caso, por sí solo, no se registra como prueba ejecutada.

6.3. Planificación y estado de actividades

El estado corresponde al 29 de octubre de 2026, fecha de la ejecución documentada.

Sprint	Semanas	Actividades previstas	Estado
1	6–7	Completar unitarias e integración y sus evidencias. Ampliar casos e iniciar funcionales.	Realizado (ver secciones 9 a 11)
2	8–9	Ejecutar funcionales, registrar incidencias y comprobar correcciones.	Realizado
3	10–11	Pruebas básicas de rendimiento y seguridad.	Realizado
4	12–13	Consolidar el segundo avance (semana 12) e iniciar pruebas móviles.	APF2 consolidado; pruebas móviles pendientes
5	14–15	Usabilidad, accesibilidad y regresión.	Pendiente
6	16–17	Tercer avance y consolidación del informe final.	Pendiente

6.4. Organización del trabajo

Las modificaciones de casos se registran con su justificación. Las correcciones de las pruebas se documentan como incidencias del instrumento (INC-T01, INC-T02). Toda regla dependiente de la versión se vincula al commit 500158f732419217507c7656904b8e6aa1bcc0d6.

7. Requisitos y trazabilidad

7.1. Requisitos funcionales

Código	Requisito
RF01	Registrar propietarios y validar los campos obligatorios y el formato del teléfono.
RF02	Buscar propietarios por apellido y consultar su información.
RF03	Actualizar los datos de un propietario existente, conservando las validaciones del registro.
RF04	Registrar mascotas asociadas a un propietario y validar datos obligatorios, nombres duplicados y fecha de nacimiento.
RF05	Actualizar el nombre, tipo y fecha de nacimiento de una mascota, respetando las validaciones correspondientes.
RF06	Registrar visitas para una mascota del propietario indicado, con descripción no vacía y fecha posterior al día actual (RN07).

7.2. Matriz de trazabilidad actualizada

La cobertura se califica respecto de las reglas definidas. «Completa» indica que todas las reglas asociadas tienen al menos un caso ejecutado; no implica ausencia de defectos.

Req.	Casos	Cobertura	Pendiente o con hallazgo
RF01	CP01; F01–F06; S03, S05, S10	Parcial	Teléfono vacío y nombre vacío en el registro (el nombre vacío se probó en la actualización, F14)
RF02	CP02; F07–F11; S04, S09	Completa para el alcance definido	Navegación de páginas posteriores a la 2
RF03	CP03; F12–F14; S06	Parcial	Dirección o ciudad vacías durante la actualización
RF04	CP04; F15–F21; E01	Completa para RN03–RN05 en flujo secuencial	RN02 falla bajo envíos simultáneos (DEF-P02, DEF-P03)
RF05	CP05; F22–F25; S08	Completa para las reglas definidas	Manejo del error al editar una mascota ajena (DEF-P01)
RF06	CP06; F26–F31; S07	Completa para RN06 y RN07	Código HTTP 500 para mascotas no asociadas (OBS-02)

Estado de la última versión: 44 casos aprobados, 6 fallidos, 1 observación y 0 bloqueados, sobre 51 casos. La matriz detallada se encuentra en matriz/APF2_Matriz_Casos_Incidencias_Metricas.xlsx.

8. Plan de pruebas

8.1. Objetivo, alcance y estrategia

Verificar los requisitos seleccionados en cinco niveles: unitario (CP01, CP05), integración con repositorios y MockMvc (CP02–CP04, CP06), funcional por HTTP real (F01–F31), carga básica local y seguridad básica (S01–S13). Se usan datos válidos, inválidos y límites confirmados en el código.

8.2. Ambiente, herramientas y datos

Elemento	Configuración usada
Sistema	Spring Petclinic, commit 500158f732419217507c7656904b8e6aa1bcc0d6; Spring Boot 4.1.0; artefacto spring-petclinic-4.0.0-SNAPSHOT.jar (SHA-256 en evidencias/AMBIENTE.txt).
Entorno	Ubuntu 24.04.5 LTS, 2 vCPU, 7,8 GiB; OpenJDK 21.0.12.1; Maven Wrapper 3.9.16. Transferencia previa: Windows, Temurin 21.0.13+9.
Base de datos	MySQL 8.0.46 local con db/mysql/schema.sql y data.sql del proyecto (usuario petclinic). H2 2.4.240 como comparación.
Herramientas	JUnit 5, Spring Boot Test, MockMvc; Python 3.13 con requests (funcionales y seguridad); Playwright 1.63 con Chromium (navegador automatizado); Locust 2.47.0 (rendimiento).
Ejecución	./mvnw test con filtros por clase; scripts en scripts/; comandos en el Anexo B.
Control de versiones	Git. Repositorio del equipo: https://github.com/Mooneer17/spring-petclinic-apf2 (rama main), con el código, los scripts, las evidencias y los documentos de este avance.
Datos	Ficticios. Cada caso funcional crea sus propios registros con un sufijo único; las pruebas JPA revierten su transacción.

8.3. Criterios de inicio y finalización

Un caso es aprobado si el obtenido coincide con todos los elementos del resultado esperado; fallido si difiere en alguno; y bloqueado si una condición impide ejecutarlo. El estado se evalúa contra el resultado esperado documentado antes de ejecutar y no se modifica después para conservar una aprobación. Se añade «Observación» para la única verificación que determina un estado real sin criterio de aprobación: la existencia de autenticación (S01). La ejecución finalizó con todos los casos ejecutados y sin casos bloqueados.

8.4. Responsables y evidencias

Cada evidencia identifica el caso, la ejecución y la base de datos. La asignación nominal de responsables de ejecución y revisión queda pendiente de registro por el equipo en la matriz (columna resaltada en la hoja «Casos»).

9. Diseño de casos de prueba

Se mantienen CP01 a CP06 con sus datos y resultados esperados del primer avance; se añaden objetivo, técnica de diseño, resultado obtenido, estado y evidencia de la última ejecución en MySQL. Los casos ampliados se describen en las secciones 11 y 13 y en el Anexo F; la hoja «Casos» de la matriz contiene todos sus campos.

CP01 / RF01 · Unitaria

Campo	Descripción
Objetivo	Comprobar que la validación rechaza un teléfono de 9 dígitos y acepta uno de 10.
Técnica de diseño	Valores límite (9 y 10 dígitos) (positivo y negativo)
Precondiciones y datos	Validador Bean Validation disponible; propietario con datos válidos salvo el teléfono. Ana Torres, Av. Central 123, Lima; 987654321 y 9876543210.
Procedimiento	Validar con Bean Validation el teléfono de 9 dígitos; repetir con 10 dígitos
Resultado esperado	9 dígitos: error en telephone; 10 dígitos: sin errores
Resultado obtenido	Coincide con lo esperado
Estado	Aprobado (R3, MySQL). Primera ejecución: Aprobado
Evidencia	evidencias/03_correccion_MySQL/surefire-reports

CP02 / RF02 · Integración (repositorio JPA)

Campo	Descripción
Objetivo	Comprobar que la búsqueda por apellido recupera el propietario almacenado con sus datos.
Técnica de diseño	Escenario positivo (positivo)
Precondiciones y datos	Contexto Spring Boot con la base de prueba (H2 o MySQL); transacción revertida al terminar; el caso crea sus datos. Propietario TorresAPF guardado en la base de prueba.
Procedimiento	Guardar, limpiar el contexto de persistencia y buscar por apellido
Resultado esperado	Un resultado con el mismo id, nombre y teléfono
Resultado obtenido	Coincide con lo esperado
Estado	Aprobado (R3, MySQL). Primera ejecución: Aprobado
Evidencia	evidencias/03_correccion_MySQL/surefire-reports

CP03 / RF03 · Integración (repositorio JPA)

Campo	Descripción
Objetivo	Comprobar que la actualización del teléfono se persiste y conserva el identificador.
Técnica de diseño	Escenario positivo (positivo)
Precondiciones y datos	Contexto Spring Boot con la base de prueba (H2 o MySQL); transacción revertida al terminar; el caso crea sus datos. Propietario existente; nuevo teléfono 9123456780.
Procedimiento	Actualizar, guardar, limpiar el contexto y recargar por id
Resultado esperado	Mismo id y teléfono 9123456780

Campo	Descripción
Resultado obtenido	Coincide con lo esperado
Estado	Aprobado (R3, MySQL). Primera ejecución: Aprobado
Evidencia	evidencias/03_correccion_MySQL/surefire-reports

CP04 / RF04 · Integración (repositorio JPA)

Campo	Descripción
Objetivo	Comprobar que una mascota registrada queda asociada al propietario con sus datos.
Técnica de diseño	Escenario positivo (positivo)
Precondiciones y datos	Contexto Spring Boot con la base de prueba (H2 o MySQL); transacción revertida al terminar; el caso crea sus datos. Mascota LunaAPF, 2022-05-10, primer tipo disponible.
Procedimiento	Asociar, guardar, limpiar el contexto y recuperar por id de propietario y de mascota
Resultado esperado	Mascota no nula con nombre, fecha y tipo registrados
Resultado obtenido	Mascota recuperada con id asignado, nombre LunaAPF, fecha 2022-05-10 y tipo no nulo
Estado	Aprobado (R3, MySQL). Primera ejecución: Fallido
Evidencia	evidencias/03_correccion_MySQL/surefire-reports; incidencia INC-T01

CP05 / RF05 · Unitaria

Campo	Descripción
Objetivo	Comprobar que PetValidator rechaza un nombre vacío y acepta uno válido.
Técnica de diseño	Partición de equivalencia (vacío / válido) (positivo y negativo)
Precondiciones y datos	Mascota con tipo cat y fecha 2022-05-10; PetValidator instanciado. Mascota con tipo cat y fecha 2022-05-10; nombre '' y 'Luna'.
Procedimiento	Ejecutar PetValidator.validate() con nombre vacío y con Luna
Resultado esperado	Vacío: error en name; Luna: sin errores (no comprueba persistencia)
Resultado obtenido	Coincide con lo esperado
Estado	Aprobado (R3, MySQL). Primera ejecución: Aprobado
Evidencia	evidencias/03_correccion_MySQL/surefire-reports

CP06 / RF06 · Integración (MockMvc + JPA)

Campo	Descripción
Objetivo	Comprobar que el flujo web registra una visita y la asocia a la mascota.
Técnica de diseño	Escenario positivo (positivo)
Precondiciones y datos	Contexto Spring Boot con la base de prueba (H2 o MySQL); transacción revertida al terminar; el caso crea sus datos. Mascota persistida; fecha de mañana; 'Control general APF'.
Procedimiento	POST /owners/{id}/pets/{petId}/visits/new con MockMvc; recargar la mascota
Resultado esperado	Redirección 3xx; una visita con la fecha y descripción enviadas
Resultado obtenido	Redirección 302; una visita con fecha de mañana y 'Control general APF'

Campo	Descripción
Estado	Aprobado (R3, MySQL). Primera ejecución: Fallido
Evidencia	evidencias/03_correccion_MySQL/surefire-reports; incidencia INC-T01

9.1. Ampliación de casos

Conjunto	Técnica de ejecución	Casos	Propósito
F01–F31	Funcional, HTTP real	31	Datos válidos, inválidos y límites de los seis flujos; verifica código HTTP, mensaje mostrado y persistencia
N01–N08	Navegador automatizado	8 capturas	Recorrido visual de los seis flujos con dos rechazos
S01–S13	Seguridad básica, HTTP real	13	Autenticación existente, configuración, validación de entradas e integridad propietario–mascota
E01	Prueba adicional de concurrencia	1	Envíos simultáneos del alta de mascota (RN02, RN04)

9.2. Técnicas de diseño aplicadas

Técnica	Casos	Ejemplos aplicados
Valores límite	11: CP01, F02, F03, F06, F18, F19, F20, F24, F26, F27, F31	Teléfono de 9, 10 y 11 dígitos; 30/31 caracteres; 255/256 caracteres; fechas de hoy y mañana
Partición de equivalencia	18: CP05, F04, F05, F07, F08, F09, F10, F11, F13, F14, F16, F17, F21, F23, F25, F28, F29, F30	Teléfono numérico frente a teléfono con letras; vacío, en blanco o ausente; duplicado; una, varias o ninguna coincidencia; fechas pasadas
Escenario positivo	9: CP02, CP03, CP04, CP06, F01, F12, F15, F22, F26	Registro, búsqueda, actualización y alta de visita con datos válidos

Un caso puede aplicar más de una técnica (F26: escenario positivo con el primer valor válido de fecha). Los casos de seguridad se diseñaron como casos negativos por control (sección 13). No se aplicaron tablas de decisión ni transición de estados.

10. Pruebas unitarias e integración

10.1. Ejecuciones realizadas

Las ejecuciones se registran por separado y no se suman entre sí. R0 corresponde al paquete recibido; R1 a R3 se ejecutaron en el ambiente consolidado.

Ejecución	Ejec.	Aprob.	Fall.	Casos fallidos	Duración Maven
Transferencia (Windows, H2)	6	4	2	CP04, CP06	26,590 s
R1 original Linux H2	6	4	2	CP04, CP06	01:46 min
R1b original MySQL	6	4	2	CP04, CP06	23,014 s
R2 corregido H2	6	6	0	-	27,330 s
R3 corregido MySQL	6	6	0	-	25,369 s

Fuente: logs y surefire-reports de evidencias/00_transferencia, 01_reproduccion_original, 01b_original_en_MySQL, 02_correccion_H2 y 03_correccion_MySQL. La duración de R1 incluye la descarga inicial de dependencias. R2 y R3 incluyen además la prueba de diagnóstico DIAG01.

10.2. Análisis de las fallas de CP04 y CP06

Las dos fallas se reprodujeron sin cambios en Linux con H2 y con MySQL; por tanto, no dependían del sistema operativo ni del motor de base de datos. En CP06, la respuesta 404 indicaba la ruta «owners/11/pets/visits/new»: el identificador de la mascota llegaba vacío.

El helper pet() de la prueba creaba una mascota, la asociaba a un propietario ya persistido y llamaba a saveAndFlush. Para una entidad existente, Spring Data ejecuta merge; la cascada MERGE persiste una copia gestionada de la mascota y la instancia creada por la prueba conserva el identificador nulo (Jakarta EE, 2022). Con ese identificador nulo, CP04 buscaba getPet(null) y CP06 construía una URL sin petId.

Para confirmar la hipótesis se añadió DIAG01, que replica el helper original y verifica que la instancia de la prueba queda con id nulo y fuera del contexto de persistencia, mientras la base de datos sí contiene la mascota con id asignado. DIAG01 aprobó en H2 y en MySQL. La causa se clasifica como problema del instrumento de prueba (INC-T01), no como defecto de Petclinic.

10.3. Corrección y comprobación

Se modificó únicamente el helper pet(): tras el flush, limpia el contexto, recarga el propietario y obtiene la mascota persistida, con dos aserciones de preparación. No se modificaron las aserciones de CP04 ni de CP06 ni el código de producción. El Anexo E reproduce el cambio.

Tras la corrección, CP01 a CP06 aprobaron en H2 (R2) y en MySQL 8.0.46 (R3). Los reportes originales se conservan en evidencias/00_transferencia y los nuevos en carpetas separadas.

10.4. Ejecución sobre MySQL

La configuración de pruebas del proyecto asocia el perfil mysql a un contenedor de Testcontainers, que requiere Docker. Al no disponer de Docker, se pasaron por línea de comandos las mismas propiedades de application-mysql.properties apuntando al MySQL local (AMB-02). El registro de R3 confirma el controlador MySQL Connector/J y el dialecto MySQLDialect.

10.5. Suite original del proyecto

Como referencia de estabilidad se ejecutó la suite completa en H2: 86 pruebas ejecutadas, 0 fallidas y 2 omitidas (MySqlIntegrationTests, por requerir Docker). Incluye las siete pruebas académicas; las demás pertenecen al proyecto y no se cuentan como trabajo del equipo.

10.6. Cobertura pendiente en este nivel

Los casos unitarios y de integración no comprueban todas las combinaciones de datos. La ampliación de validaciones, duplicados, fechas y edición se realizó en el nivel funcional (sección 11).

11. Pruebas funcionales

11.1. Enfoque y técnicas

Spring Petclinic es una aplicación web con formularios HTML renderizados en el servidor; no expone una API REST con JSON ni está construida con microservicios. Por eso las pruebas no validan contratos JSON (UTP, 2026d), sino formularios, redirecciones y páginas HTML. Se usaron tres técnicas de ejecución, que no deben confundirse:

Técnica	Descripción	Casos
Solicitudes HTTP reales	Script Python que envía los formularios por HTTP a la aplicación en ejecución (puerto 8080, MySQL). En cada caso verifica el código HTTP, el mensaje mostrado (error del campo o mensaje de éxito) y, cuando corresponde, los datos persistidos en la página de detalle.	F01–F31
Navegador automatizado	Chromium controlado con Playwright: completa formularios y guarda capturas. Reproduce acciones de usuario, pero no equivale a una prueba manual.	N01–N08
MockMvc	Simula solicitudes dentro del contexto de Spring, sin servidor HTTP.	CP06
Prueba manual	No realizada en esta ejecución.	Pendiente del equipo

11.2. Precondiciones

Aplicación iniciada con el perfil mysql sobre MySQL 8.0.46 con los datos semilla del proyecto. Cada caso crea sus propios propietarios y mascotas con un sufijo único, de modo que ningún caso depende de otro. Las fechas son relativas al día de ejecución (hoy, mañana y ayer). Los mensajes esperados se tomaron del código evaluado: anotaciones de validación, PetValidator, controladores y messages.properties.

11.3. Resultados

La tabla resume la corrida 3 (30/10/2026), que verifica código, mensaje y persistencia. Los pasos numerados, las precondiciones y la ruta de cada respuesta HTTP guardada figuran en el Anexo F, en la matriz y en evidencias/05c_funcionales_MySQL_corrida3_v2/respuestas.

ID	Caso y datos	Técnica	Esperado	Obtenido	Estado
F01 RF01	Registro de propietario con datos válidos. Datos: Ana Torres, Av. Central 123, Lima, 9876543210	Escenario positivo	302 a /owners/{id}; mensaje 'New Owner Created'; el detalle muestra los datos registrados	HTTP 302, id=179; mensaje='New Owner Created'; datos=sí	Aprobado
F02 RF01	Registro rechazado: teléfono de 9 dígitos. Datos: teléfono='987654321'	Valores límite (justo debajo: 9)	200 con 'Telephone must be a 10-digit number' en telephone; el propietario no se guarda	HTTP 200; mensaje en telephone='Telephone must be a 10-digit number'; no guardado=sí	Aprobado
F03 RF01	Registro rechazado: teléfono de 11 dígitos. Datos: teléfono='98765432101'	Valores límite (justo encima: 11)	200 con 'Telephone must be a 10-digit number' en telephone; el propietario no se guarda	HTTP 200; mensaje en telephone='Telephone must be a 10-digit number'; no guardado=sí	Aprobado
F04 RF01	Registro rechazado: teléfono con letras. Datos: teléfono='98765abcde'	Partición de equivalencia (clase no numérica)	200 con 'Telephone must be a 10-digit number' en telephone; el propietario no se guarda	HTTP 200; mensaje en telephone='Telephone must be a 10-digit number'; no guardado=sí	Aprobado
F05 RF01	Registro rechazado: dirección vacía y ciudad en blanco. Datos: address='', city=' '	Partición de equivalencia (vacío y en blanco)	200 con 'must not be blank' en address y city; no se guarda	HTTP 200; mensaje en address='must not be blank'; mensaje en city='must not be blank'; no guardado=sí	Aprobado
F06 RF01	Límite de longitud del apellido (30/31). Datos: apellido de 30 y de 31 caracteres	Valores límite (30 / 31)	30: 302; 31: 200 con 'size must be between 0 and 30' en lastName	30 -> HTTP 302; 31 -> HTTP 200, mensaje en lastName='size must be between 0 and 30'	Aprobado

ID	Caso y datos	Técnica	Esperado	Obtenido	Estado
F07 RF02	Búsqueda con coincidencia única. Datos: apellido=FgSXYJY	Partición de equivalencia (coincidencia única)	302 al detalle del propietario {id}, que muestra su apellido	HTTP 302 -> /owners/181; apellido en detalle=sí	Aprobado
F08 RF02	Búsqueda por prefijo con varias coincidencias. Datos: prefijo=FhSXYJY (2 propietarios)	Partición de equivalencia (varias coincidencias)	200 con lista que incluye ambos propietarios	HTTP 200; ambos listados=sí	Aprobado
F09 RF02	Búsqueda sin resultados. Datos: apellido=InexistenteSXYJY	Partición de equivalencia (sin coincidencias)	200 con mensaje 'has not been found'	HTTP 200; mensaje presente=sí	Aprobado
F10 RF02	Búsqueda con espacios al inicio y al final. Datos: apellido=' FjSXYJY '	Partición de equivalencia (entrada con espacios)	Se recortan los espacios: 302 al detalle del propietario creado	HTTP 302 -> /owners/184	Aprobado
F11 RF02	Búsqueda con apellido vacío (todos). Datos: lastName=''	Partición de equivalencia (entrada vacía)	200 con listado paginado de propietarios	HTTP 200; listado paginado=sí	Aprobado
F12 RF03	Actualización válida de teléfono y ciudad. Datos: teléfono 9123456780, ciudad Arequipa	Escenario positivo	302 al mismo propietario; mensaje 'Owner Values Updated'; se muestran los nuevos valores	HTTP 302 -> /owners/185; mensaje='Owner Values Updated'; persistido=sí	Aprobado
F13 RF03	Actualización rechazada: teléfono de 5 dígitos. Datos: teléfono=12345	Partición de equivalencia (longitud inválida)	200 con 'Telephone must be a 10-digit number'; se conserva el teléfono anterior	HTTP 200; mensaje en telephone='Telephone must be a 10-digit number'; teléfono conservado=sí	Aprobado
F14 RF03	Actualización rechazada: nombre vacío. Datos: firstName=''	Partición de equivalencia (vacío)	200 con 'must not be blank' en firstName; datos sin cambios	HTTP 200; mensaje en firstName='must not be blank'; nombre conservado=sí	Aprobado
F15 RF04	Registro de mascota válida. Datos: Luna, 2022-05-10, cat	Escenario positivo	302; mensaje 'New Pet has been Added'; la mascota aparece asociada con nombre, fecha y tipo	HTTP 302; mensaje='New Pet has been Added'; petId=261; datos=sí	Aprobado
F16 RF04	Registro rechazado: nombre vacío. Datos: name=''	Partición de equivalencia (vacío)	200 con 'is required' en name; no se registra la mascota	HTTP 200; mensaje en name='is required'; sin mascota registrada=sí	Aprobado
F17 RF04	Registro rechazado: nombre duplicado (distinta capitalización). Datos: Luna existente; nueva 'LUNA'	Partición de equivalencia (nombre duplicado)	200 con 'is already in use' en name; sigue existiendo una sola mascota	HTTP 200; mensaje en name='is already in use'; mascotas con ese nombre=1	Aprobado
F18 RF04	Registro rechazado: nacimiento futuro (mañana). Datos: birthDate=2026-10-31	Valores límite (mañana)	200 con 'invalid date' en birthDate; no se registra la mascota	HTTP 200; mensaje en birthDate='invalid date'; sin mascota registrada=sí	Aprobado
F19 RF04	Límite: fecha de nacimiento igual a hoy. Datos: birthDate=2026-10-30	Valores límite (hoy)	302 y mensaje 'New Pet has been Added' (la regla solo rechaza fechas posteriores a hoy)	HTTP 302; mensaje='New Pet has been Added'	Aprobado
F20 RF04	Límite de longitud del nombre de mascota (30/31). Datos: nombre de 30 y de 31 caracteres	Valores límite (30 / 31)	30: 302; 31: 200 con 'Name must be no more than 30 characters'	30 -> HTTP 302; 31 -> HTTP 200, mensaje en name='Name must be no more than 30 characters'	Aprobado
F21 RF04	Registro rechazado: sin fecha de nacimiento. Datos: birthDate=''	Partición de equivalencia (ausente)	200 con 'is required' en birthDate; no se registra la mascota	HTTP 200; mensaje en birthDate='is required'; sin mascota registrada=sí	Aprobado
F22 RF05	Edición válida de nombre, tipo y fecha. Datos: Luna->Lunita, cat->dog, 2021-03-15	Escenario positivo	302; mensaje 'Pet details has been edited'; la misma mascota (mismo id) muestra los nuevos datos	HTTP 302; mensaje='Pet details has been edited'; mismo id=sí; datos=sí	Aprobado
F23 RF05	Edición rechazada: renombrar a un nombre ya usado. Datos: Max -> Luna (existente)	Partición de equivalencia (nombre duplicado)	200 con 'is already in use' en name; Max conserva su nombre	HTTP 200; mensaje en name='is already in use'; Max sin cambios=sí	Aprobado
F24 RF05	Edición rechazada: fecha de nacimiento futura. Datos: birthDate=2026-10-31	Valores límite (mañana)	200 con 'invalid date' en birthDate; se conserva la fecha anterior	HTTP 200; mensaje en birthDate='invalid date'; fecha conservada=sí	Aprobado
F25 RF05	Edición rechazada: nombre vacío (persistencia, complementa CP05). Datos: name=''	Partición de equivalencia (vacío)	200 con 'is required' en name; la mascota conserva su nombre	HTTP 200; mensaje en name='is required'; nombre conservado=sí	Aprobado
F26 RF06	Registro de visita válida (mañana). Datos: fecha=2026-10-31, 'Control general'	Escenario positivo; valores límite (mañana)	302; mensaje 'Your visit has been booked'; la visita aparece en la mascota	HTTP 302; mensaje='Your visit has been booked'; visita visible=sí	Aprobado
F27 RF06	Visita rechazada: fecha de hoy (límite). Datos: fecha=2026-10-30	Valores límite (hoy)	200 con 'Visit date must be in the future' en date; no se registra la visita	HTTP 200; mensaje en date='Visit date must be in the future'; no registrada=sí	Aprobado
F28 RF06	Visita rechazada: fecha pasada. Datos: fecha=2026-10-29	Partición de equivalencia (fechas pasadas)	200 con 'Visit date must be in the future' en date; no se registra la visita	HTTP 200; mensaje en date='Visit date must be in the future'; no registrada=sí	Aprobado

ID	Caso y datos	Técnica	Esperado	Obtenido	Estado
F29 RF06	Visita rechazada: descripción vacía. Datos: descripción=''	Partición de equivalencia (vacío)	200 con 'must not be blank' en description; no se registra la visita	HTTP 200; mensaje en description='must not be blank'; no registrada=sí	Aprobado
F30 RF06	Visita rechazada: descripción solo con espacios. Datos: descripción=' '	Partición de equivalencia (en blanco)	200 con 'must not be blank' en description; no se registra la visita	HTTP 200; mensaje en description='must not be blank'; no registrada=sí	Aprobado
F31 RF06	Límite de longitud de la descripción (255/256). Datos: descripción de 255 y de 256 caracteres	Valores límite (255 / 256)	255: 302 y 'Your visit has been booked'; 256: 200 con 'size must be between 0 and 255'	255 -> HTTP 302, mensaje='Your visit has been booked'; 256 -> HTTP 200, mensaje en description='size must be between 0 and 255'	Aprobado

Fuente: evidencias/05c_funcionales_MySQL_corrida3_v2/resultados.json. Corridas anteriores (versión 1 del script): corrida 1 con 29 aprobados y 2 fallidos (INC-T02); corrida 2 con 31 aprobados, sin verificación del texto de los mensajes (INC-T03).

11.4. Recorrido en navegador automatizado

Las capturas N01 a N08 recorren los seis flujos: registro rechazado por teléfono de nueve dígitos y registro válido, búsqueda, actualización del teléfono, alta de mascota, rechazo de fecha de nacimiento futura, edición de mascota y registro de visita.

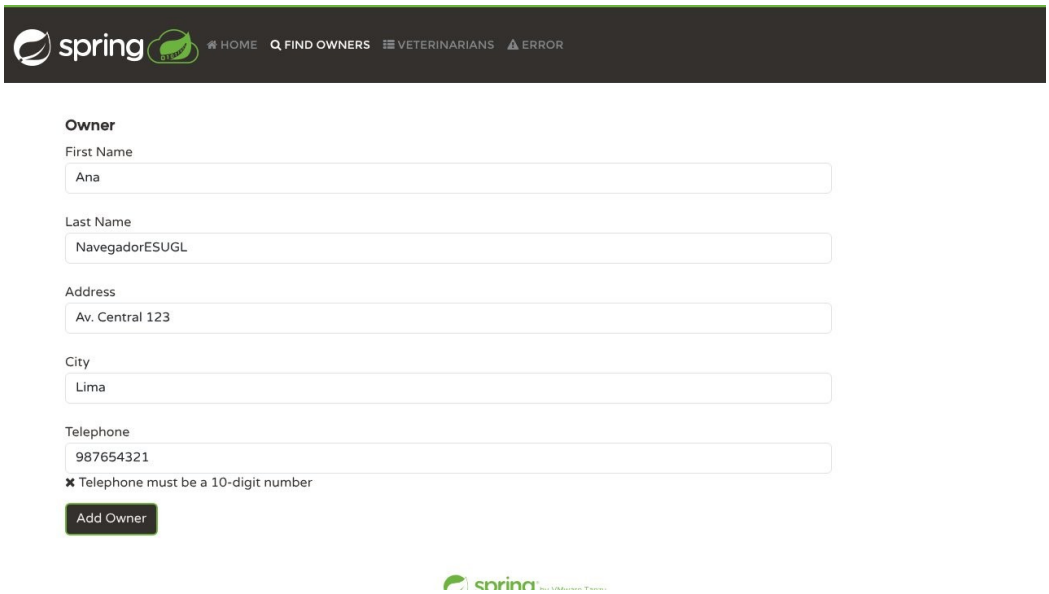


Figura 1. N01: el formulario devuelve el error «Telephone must be a 10-digit number» (RF01).

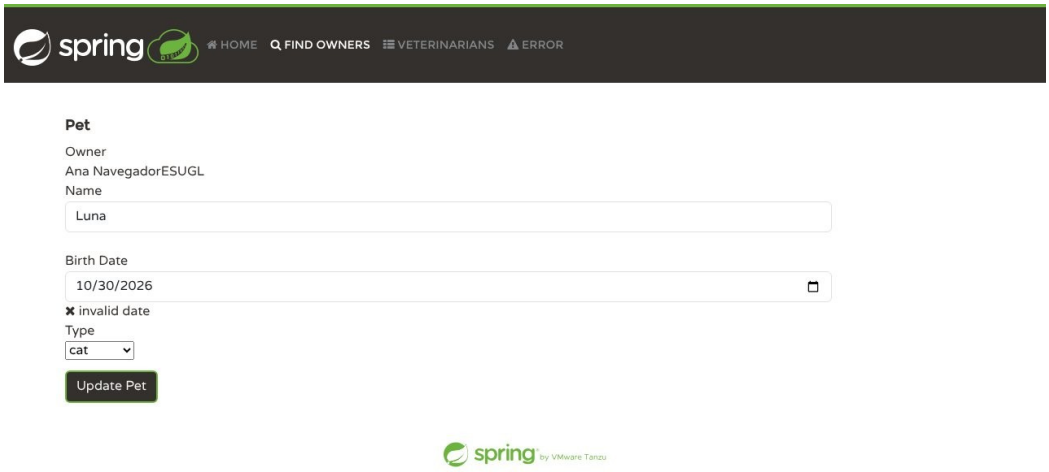


Figura 2. N06: la edición con fecha de nacimiento de mañana se rechaza con «invalid date» (RF05).

11.5. Prueba adicional de concurrencia (E01)

Clasificación. E01 se registra como prueba adicional de concurrencia. No se clasifica como prueba exploratoria porque no se definió de antemano un charter ni un timebox, y no se llevó una bitácora de acciones y decisiones ni un debrief, elementos que exige una sesión exploratoria (UTP, 2026b). Lo que se conserva son los resultados de cada ejecución, las consultas a la base de datos y la traza SQL. El escenario coincide con la heurística de repetir acciones rápidamente, pero eso no basta para documentarlo como sesión. Tampoco es una prueba de estrés: no aumenta la demanda para buscar un límite, sino que envía cuatro solicitudes simultáneas por propietario.

Procedimiento documentado. Para cada propietario nuevo se enviaron en paralelo cuatro altas de la mascota «Gemela», situación equivalente a un doble envío del formulario. Esperado: una mascota por propietario, rechazo controlado de las demás solicitudes, ninguna mascota sin propietario y ningún HTTP 500.

Ejecución	Propietarios	Respuestas 302	Rechazo controlado	HTTP 500	Mascotas sin propietario
MySQL, corrida 1	10	12	0	28	2
MySQL, corrida 2	20	27	2	51	7
H2 (modo archivo)	20	38	42	0	18

Fuente: resultado.json de evidencias/07, 07b y 07c; conteo de filas con owner_id nulo consultado directamente en cada base.

Hallazgos. Cada respuesta 302 adicional a la primera dejó una mascota desvinculada. La traza SQL de MySQL (evidencias/07d) registra que la segunda solicitud ejecuta «update pets set owner_id=null where owner_id=… and id=…»: al fusionar su copia del propietario, desvincula la mascota que otra solicitud acababa de confirmar. El usuario recibe un mensaje de éxito para un registro que deja de ser visible, lo que incumple RN02 (DEF-P02). En MySQL, además, la violación de unicidad se informa con la clave «pets.owner_id», mientras el controlador reconoce solo «unique_owner_pet_name», nombre definido en el esquema H2; por ello la solicitud duplicada produce HTTP 500 en lugar del mensaje de validación (DEF-P03). E01 se registra como fallido.

Limitaciones. La causa propuesta (ausencia de control de concurrencia sobre el propietario) se apoya en la traza SQL, pero no se verificó con una corrección. Los conteos dependen de la temporización de cada ejecución y pueden variar entre equipos.

11.6. Análisis

En flujo secuencial, los 31 casos funcionales coinciden con las reglas RN01 a RN07: se rechazan teléfonos de 9 y 11 dígitos o con letras, campos vacíos, nombres duplicados aun con distinta capitalización, fechas futuras de nacimiento y visitas de hoy o pasadas, y en cada rechazo se muestra el mensaje previsto. Los límites de 30 caracteres (apellido y nombre de mascota) y 255 (descripción) se aceptan, y uno más se rechaza. La fecha de nacimiento igual a hoy se acepta, coherente con RN05.

Los defectos aparecen fuera del flujo secuencial: en envíos simultáneos (E01) y en el manejo de identificadores no asociados (sección 13).

12. Pruebas básicas de carga local

12.1. Objetivo y escenario

El objetivo fue obtener una línea base de tiempos de respuesta y errores bajo una carga moderada y conocida. Se trata de pruebas básicas de carga local: no determinan la capacidad máxima del sistema ni constituyen una prueba de estrés. No existe una demanda real documentada para una clínica; por ello, el número de usuarios y los umbrales son decisiones académicas del equipo.

Se modeló el trabajo de recepción de la clínica con cuatro operaciones y pesos relativos: búsqueda de propietarios por apellido (4), consulta del detalle de un propietario semilla (4), apertura del formulario de búsqueda (1) y registro de una visita en una mascota propia de cada usuario virtual (1). La distribución esperada de las acciones es 40 %, 40 %, 10 % y 10 %. Al iniciar, cada usuario virtual crea un propietario y una mascota; esas solicitudes de preparación se reportan por separado.

12.2. Configuración

Parámetro	Valor
Herramienta	Locust 2.47.0, modo sin interfaz (scripts/locustfile.py); compatible con el material de clase (UTP, 2026c)
Aplicación y entorno	JAR del commit evaluado, perfil mysql, JVM -Xmx512m, reinicio previo; MySQL 8.0.46; contenedor Linux de 2 vCPU y 7,8 GiB compartido por aplicación, base de datos y generador
Usuarios virtuales (VU)	Escenario A: 10 VU. Escenario B: 30 VU
Incorporación gradual (ramp-up)	A: 2 VU/s (10 VU en unos 5 s). B: 3 VU/s (30 VU en unos 10 s)
Duración sostenida	180 s por escenario (A: 19:19:47 a 19:22:47; B: 19:22:56 a 19:25:57 del 29/10/2026)
Retiro (ramp-down)	No configurado: Locust detiene todos los usuarios al final
Pausas (think time)	Entre 1 y 2 s aleatorios entre acciones de cada usuario
Calentamiento	60 s con 10 VU antes del escenario A; guardado y excluido de las métricas
Métricas	Solicitudes, errores (tasa de error), mediana, promedio, p95, p99, máximo y throughput (solicitudes/s); memoria residente del proceso Java
Umbrales del equipo	p95 < 1000 ms y tasa de error < 1 %. Son decisiones académicas, no exigencias del docente ni estándares universales
Repeticiones	Una ejecución por escenario

12.3. Resultados

Escenario	Solicitudes	Errores	Mediana	Promedio	p95	p99	Máximo	Solic./s
A: 10 VU	1212	0 (0 %)	11 ms	12 ms	21 ms	29 ms	87 ms	6,74
B: 30 VU	3592	0 (0 %)	8 ms	8 ms	16 ms	26 ms	86 ms	19,96

Fuente: resumen final de consola.log de Locust en evidencias/08_rendimiento_MySQL. Incluye las solicitudes de preparación. Detalle por operación en la hoja «Rendimiento» de la matriz y en reporte.html.

Operación	Solic. (A)	p95 (A)	Solic. (B)	p95 (B)	Máx. (B)
GET /owners?lastName	464	14 ms	1378	11 ms	38 ms
GET /owners/[id]	457	20 ms	1418	15 ms	86 ms
GET /owners/find	127	11 ms	356	9 ms	22 ms
POST /visits/new	134	28 ms	350	19 ms	30 ms

12.4. Interpretación y limitaciones

No se registraron errores en 4804 solicitudes medidas. El throughput creció de 6,74 a 19,96 solicitudes por segundo, en proporción al número de usuarios, y el p95 se mantuvo entre 16 y 21 ms. Ambos escenarios cumplen los umbrales definidos por el equipo.

El p95 fue menor con 30 usuarios que con 10. Este resultado no se atribuye al aumento de carga. Una hipótesis es que la JVM y las cachés de la aplicación y de la base de datos seguían optimizándose durante el escenario A, pese al calentamiento previo; no se verificó con mediciones adicionales, por lo que se presenta solo como hipótesis. La memoria residente del proceso Java se mantuvo entre 386 y 390 MiB durante el escenario B.

Limitaciones: cada escenario se ejecutó una sola vez; el generador compartió el equipo con la aplicación y la base de datos, lo que puede influir en los tiempos; y los datos semilla son pocos. La prueba no determina la capacidad máxima, el comportamiento con red real ni con datos voluminosos, y no sustituye una prueba en un entorno equivalente a producción.

13. Pruebas de seguridad básica

13.1. Alcance y tipo de prueba

Se realizó un análisis básico de vulnerabilidades sobre la aplicación en ejecución, solo en la instancia local propia (el script rechaza otros hosts). No incluyó fuerza bruta, denegación de servicio ni herramientas de explotación. Por su alcance, no constituye una prueba de penetración ni una auditoría de aplicaciones (UTP, 2026e). Las verificaciones se agrupan en cinco categorías de control: autenticación, autorización, integridad de la relación propietario–mascota, validación de entradas y configuración de seguridad.

13.2. Autenticación y autorización

El pom.xml no incluye Spring Security y no existe una ruta de inicio de sesión (/login responde 404). Todas las rutas de consulta y modificación responden 200 a solicitudes anónimas. Por tanto, no hay cuentas, roles ni permisos: no existen pruebas de autorización aplicables y no se añadió un módulo de autenticación, porque no forma parte de RF01–RF06. S01 se registra como observación de alcance.

Las verificaciones S07 y S08 no son pruebas de autorización. Comprueban la integridad de la relación entre un propietario registrado y sus mascotas, es decir, que una operación dirigida a un propietario no altere datos de otro. Sin usuarios autenticados no puede evaluarse si un usuario tiene permiso sobre un recurso.

13.3. Resultados

ID	Categoría	Verificación	Esperado	Obtenido	Estado	Incid.
S01	Autenticación	Verificación de autenticación existente	Determinar si la variante exige autenticación (no se asume un resultado)	/=200; /owners/find=200; /owners?lastName==200; /owners/1=200; /owners/1/edit=200; /vets.html=200; /owners/new=200; /login=404	Observación	OBS-01
S02	Configuración de seguridad	Exposición de endpoints de administración (Actuator)	Endpoints sensibles no accesibles de forma anonima	/actuator=200; /actuator/env=200; /actuator/configprops=200; /actuator/beans=200; /actuator/mappings=200; /actuator/heapdump(HEAD)=404	Fallido	SEC-01
S03	Validación de entradas	XSS almacenado en dirección y ciudad	El contenido se muestra escapado; no aparece una etiqueta <script> ejecutable	etiqueta sin escapar=no; escapada=sí	Aprobado	-
S04	Validación de entradas	Inyección SQL en lastName	Se tratan como texto: 200 'has not been found', sin trazas ni efectos	[' OR '1'='1] HTTP 200 not_found=sí; [x%' OR 1=1 --] HTTP 200 not_found=sí; [" ; DROP TABLE owners; --] HTTP 200 not_found=sí; tabla owners operativa=sí	Aprobado	-
S05	Validación de entradas (parámetros no permitidos)	Asignación masiva: id=1 enviado al crear propietario	Se crea un propietario nuevo; el propietario 1 no se modifica	HTTP 302; nuevo id=73; propietario 1 sin cambios=sí	Aprobado	-
S06	Validación de entradas (parámetros no permitidos)	Manipulación del id en la actualización del propietario	El propietario B no se modifica	HTTP 302 -> http://localhost:8080/owners/74; B modificado=no; A modificado=sí	Aprobado	-
S07	Integridad propietario–mascota	Visita a una mascota que no pertenece al propietario de la URL	Se rechaza la operación; no se registra la visita; sin traza interna	GET HTTP 500; POST HTTP 500; visita registrada=no; traza=no	Aprobado	OBS-02
S08	Integridad propietario–mascota	Edición de una mascota que pertenece a otro propietario	Se rechaza con un error controlado; la mascota de B no cambia; no se crean registros	GET HTTP 500; POST HTTP 500; mascota B modificada=no; A recibio mascota=no; traza=no	Fallido	DEF-P01
S09	Configuración de seguridad (manejo de errores)	Propietario inexistente y divulgación en la página de error	Error controlado sin traza de pila	HTTP 500; traza=no; mensaje interno visible=no	Aprobado	OBS-02
S10	Validación de entradas	Entradas sobredimensionadas	200 con error de validación; sin error 500 ni almacenamiento	256 -> 200/True; ciudad 81 -> 200/True; 20000 -> 200/True	Aprobado	-

ID	Categoría	Verificación	Esperado	Obtenido	Estado	Incid.
S11	Configuración de seguridad (CSRF)	Protección CSRF en formularios	Si la aplicación protegiera contra CSRF, el POST sin token se rechazaria	token en formulario=no; POST HTTP 302	Fallido	SEC-02
S12	Configuración de seguridad (cabeceras)	Cabeceras de seguridad HTTP en la página principal	Presencia de cabeceras defensivas básicas	X-Content-Type-Options=no; X-Frame-Options=no; Content-Security-Policy=no; Cache-Control=no	Fallido	SEC-03
S13	Configuración de seguridad (sesión)	Identificador de sesión expuesto en la URL de redirección	La redirección no incluye el identificador de sesión	HTTP 302; Location contiene jsessionid=sí	Fallido	SEC-04

Fuente: evidencias/06_seguridad_MySQL/resultados.json y respuestas HTTP guardadas (ejecución única del 29/10/2026). S08 se reclasificó en la revisión del 30/10/2026 sin nueva ejecución (sección 17.6).

13.4. Análisis por categoría

Validación de entradas. Thymeleaf escapa el contenido almacenado (S03). La búsqueda trata las cargas de inyección SQL como texto (S04). La lista de campos no enlazables impide fijar el id al crear o actualizar un propietario (S05, S06). Las entradas sobredimensionadas se rechazan con error de validación (S10).

Integridad propietario–mascota. No fue posible registrar una visita (S07) ni modificar una mascota de otro propietario (S08): los datos se conservan. Sin embargo, S08 es fallido: su resultado esperado exige un rechazo con error controlado y la aplicación falla con NullPointerException, que produce HTTP 500 con cuerpo vacío (DEF-P01). En S07 la operación se rechaza mediante una excepción lanzada a propósito por el controlador; el código 500, en lugar de 404, se registra en OBS-02.

Configuración de seguridad. Los endpoints de Actuator están expuestos sin autenticación (S02, SEC-01): /actuator/env enmascara los valores y heapdump no está disponible, pero se publican beans, mapeos, configuración y volcado de hilos. Tampoco hay protección CSRF (S11, SEC-02), faltan cabeceras defensivas (S12, SEC-03) y la primera redirección incluye el identificador de sesión (S13, SEC-04). Ante un propietario inexistente (S09), la aplicación detecta la condición y muestra una página de error sin trazas internas, aunque con código 500 (OBS-02).

13.5. Hallazgos: impacto, severidad y recomendación

La severidad considera el impacto sobre confidencialidad, integridad o disponibilidad, la probabilidad y la exposición (UTP, 2026e). Las recomendaciones no se implementaron: el alcance excluye modificar el código de producción.

ID	Evidencia	Impacto	Severidad y justificación	Recomendación	Estado
DEF-P01	S08: 06_seguridad_MySQL/respuestas/S08_*.http.txt; registro del servidor	Disponibilidad parcial del flujo y experiencia del usuario: la petición termina en error del servidor sin mensaje útil. No se alteran datos (integridad conservada).	Media. Media: la URL es accesible por cualquiera y el fallo es reproducible, pero no compromete datos.	Propuesta, no aplicada (no se modifica código de producción): validar el resultado de findPet() y lanzar una excepción asociada a 404	Abierto
SEC-01	S02: 06_seguridad_MySQL/respuestas/S02_*	Confidencialidad: expone configuración, mapeos, componentes y volcado de hilos que facilitan el reconocimiento del sistema.	Media (local); alta si se despliega así. Media en la instancia local; alta si se despliega con esta configuración en una red accesible.	Propuesta: limitar management.endpoints. web.exposure.include a health,info fuera de desarrollo	Abierto
SEC-02	S11: 06_seguridad_MySQL/respuestas/S11_*	Integridad: una página externa puede provocar altas o cambios no solicitados en nombre del navegador del usuario.	Media. Media: el impacto existe aunque no haya cuentas; depende de que la aplicación sea accesible desde el navegador de la víctima.	Propuesta: incorporar protección CSRF si la aplicación se despliega con usuarios	Abierto
SEC-03	S12: 06_seguridad_MySQL/respuestas/S12_*	Defensa en profundidad: sin cabeceras, el navegador no aplica restricciones adicionales (por ejemplo, contra encuadre o detección de tipos).	Baja. Baja: no se identificó una explotación concreta; es una medida preventiva.	Propuesta: añadir cabeceras defensivas	Abierto

ID	Evidencia	Impacto	Severidad y justificación	Recomendación	Estado
SEC-04	S13: 06_seguridad_MySQL/respuestas/S13_*	Confidencialidad de la sesión: el identificador puede quedar en registros o historial.	Baja. Baja: la sesión no protege cuentas porque no hay autenticación.	Propuesta: server.servlet.session.tracking-modes=cookie	Abierto
OBS-01	S01: 06_seguridad_MySQL/respuestas/S01_*	Alcance: sin autenticación ni autorización, cualquier persona con acceso de red puede consultar y modificar datos.	No aplica. No aplica: es una característica de la variante evaluada, no un defecto frente a RF01–RF06.	No se añade un módulo de login; no forma parte de los requisitos RF01–RF06	Registrado
OBS-02	S07, S09: 06_seguridad_MySQL/respuestas/S07_*, S09_*	Semántica HTTP: recursos inexistentes o no asociados responden 500 en lugar de 404; dificulta el diagnóstico y el monitoreo.	Baja. Baja: la integridad se conserva y no se exponen trazas.	Propuesta: mapear la excepción a 404	Abierto

Estos resultados no permiten afirmar que el sistema sea seguro. Indican qué controles básicos existen en esta variante de demostración y cuáles deberían añadirse antes de un despliegue con usuarios reales.

14. Pruebas móviles, de usabilidad y accesibilidad

Corresponden al sprint 5 y al tercer avance. No se presentan resultados en este documento.

15. Pruebas de regresión

Corresponden al sprint 5. En este avance solo se reejecutaron los casos corregidos para comprobar las correcciones (secciones 10.3 y 16).

16. Registro de incidencias y correcciones

Las incidencias se separan en cinco categorías: defectos del producto (DEF), hallazgos de seguridad (SEC), observaciones de alcance y calidad (OBS), problemas del instrumento de prueba (INC-T) y problemas del ambiente (AMB). Solo se corrigieron los instrumentos de prueba y el ambiente. Los defectos del producto y los hallazgos de seguridad quedan abiertos porque el alcance excluye modificar el código de producción; se incluye una recomendación no implementada.

ID	Categoría	Casos	Req.	Severidad	Prioridad	Estado
INC-T01	Problema del instrumento de prueba	CP04, CP06	RF04, RF06	Alta (invalida dos casos)	Alta	Cerrado
INC-T02	Problema del instrumento de prueba	F01, F11	RF01, RF02	Media	Alta	Cerrado
INC-T03	Problema del instrumento de prueba	F01–F31	RF01–RF06	Media	Alta	Cerrado
DEF-P01	Defecto del producto (manejo de errores)	S08	RF05	Media	Media	Abierto
DEF-P02	Defecto del producto (integridad bajo concurrencia)	E01	RF04 (RN02, RN04)	Alta	Media	Abierto
DEF-P03	Defecto del producto (manejo de errores, solo MySQL)	E01	RF04 (RN04)	Media	Media	Abierto
SEC-01	Hallazgo de seguridad (configuración)	S02	Transversal	Media (local); alta si se despliega así	Media	Abierto
SEC-02	Hallazgo de seguridad (control ausente)	S11	Transversal	Media	Baja	Abierto
SEC-03	Hallazgo de seguridad (configuración)	S12	Transversal	Baja	Baja	Abierto
SEC-04	Hallazgo de seguridad (configuración)	S13	Transversal	Baja	Baja	Abierto
OBS-01	Observación de alcance	S01	Transversal	No aplica	No aplica	Registrado
OBS-02	Hallazgo de calidad (manejo de errores)	S07, S09	RF02, RF06	Baja	Baja	Abierto
AMB-01	Problema de ambiente	Todos	-	Alta (bloqueante)	Alta	Cerrado
AMB-02	Problema de ambiente	CP02–CP04, CP06	-	Media	Alta	Cerrado (mitigado)
AMB-03	Problema de ambiente	Todos (Windows)	-	Media	Alta	Cerrado

Detalle completo de reproducción, esperado, obtenido, impacto, corrección o recomendación y comprobación en la hoja «Incidencias» de la matriz. Los hallazgos de seguridad se detallan en la sección 13.5.

En el alcance funcional secuencial (F01–F31) no se identificaron defectos del producto; las fallas y limitaciones de sus primeras corridas correspondieron al script (INC-T02, INC-T03).

INC-T01 · Problema del instrumento de prueba

Campo	Descripción
Caso / requisito	CP04, CP06 / RF04, RF06
Reproducción	Ejecutar AvanceIntegrationTests original. El helper pet() devuelve la instancia Pet creada por el test. Como el propietario ya existe, saveAndFlush ejecuta merge y la cascada persiste una copia gestionada; la instancia del test conserva id nulo.
Esperado	El fixture entrega una mascota con el id asignado por la base de datos

Campo	Descripción
Obtenido	CP04: getPet(null) devuelve nulo. CP06: URL /owners/{id}/pets//visits/new y HTTP 404 ('No static resource owners/11/pets/visits/new')
Severidad / prioridad	Alta (invalida dos casos) / Alta
Estado	Cerrado
Corrección	pet() recarga la mascota desde la base tras flush y clear y verifica que tenga id; las aserciones de CP04 y CP06 no se modificaron
Comprobación posterior	R2 (H2) y R3 (MySQL): CP01–CP06 aprobados. DIAG01 confirma que la instancia original queda con id nulo
Evidencia	01_reproduccion_original, 01b_original_en_MySQL, 02_correccion_H2, 03_correccion_MySQL

INC-T02 · Problema del instrumento de prueba

Campo	Descripción
Caso / requisito	F01, F11 / RF01, RF02
Reproducción	Primera corrida de funcionales_http.py. F01: la primera redirección incluye ';jsessionid=...' y la expresión regular no la admitía. F11: el script buscaba un marcador inexistente en el listado.
Esperado	El script reconoce la redirección y el listado paginado que la aplicación sí devuelve
Obtenido	F01 y F11 marcados como fallidos aunque la aplicación respondió correctamente (302 a /owners/20; tabla id='owners' con páginas)
Severidad / prioridad	Media / Alta
Estado	Cerrado
Corrección	Se admite ';jsessionid' al extraer el id y se verifica la tabla id='owners' y el enlace a la página 2
Comprobación posterior	Corrida 2: 31 de 31 casos aprobados
Evidencia	05a_funcionales_MySQL_corrida1, 05b_funcionales_MySQL_corrida2

INC-T03 · Problema del instrumento de prueba

Campo	Descripción
Caso / requisito	F01–F31 / RF01–RF06
Reproducción	Revisión del 30/10/2026: la versión 1 de funcionales_http.py verificaba el código HTTP, la marca de error del campo y la persistencia, pero no el texto del mensaje. Además, no seguía de inmediato las redirecciones de preparación, por lo que el mensaje de éxito de una operación previa podía mostrarse en otro caso (en la corrida 2, F12 mostró 'New Owner Created').
Esperado	Cada caso verifica el mensaje exacto (error de campo o éxito), además del código HTTP y la persistencia
Obtenido	Verificación de mensajes incompleta en las corridas 1 y 2; ningún resultado de la aplicación era incorrecto
Severidad / prioridad	Media / Alta
Estado	Cerrado
Corrección	Versión 2 del script: compara el mensaje esperado tomado del código evaluado y consume cada redirección al momento
Comprobación posterior	Corrida 3 (30/10/2026, MySQL): 31 de 31 aprobados con verificación de mensaje y persistencia

Campo	Descripción
Evidencia	05c_funcionales_MySQL_corrida3_v2; scripts/funcionales_http_v1.py y funcionales_http.py

DEF-P01 · Defecto del producto (manejo de errores)

Campo	Descripción
Caso / requisito	S08 / RF05
Reproducción	GET o POST /owners/{A}/pets/{idMascotaDeB}/edit. findPet() devuelve null cuando la mascota no pertenece al propietario de la URL.
Esperado	Rechazo controlado (por ejemplo, 404), como hace VisitController al validar la pertenencia
Obtenido	HTTP 500 con cuerpo vacío. GET: error de plantilla (EL1012E, pet nulo). POST: NullPointerException en PetController.java:148. La mascota de B no se modifica
Impacto	Disponibilidad parcial del flujo y experiencia del usuario: la petición termina en error del servidor sin mensaje útil. No se alteran datos (integridad conservada).
Severidad / prioridad	Media / Media. Media: la URL es accesible por cualquiera y el fallo es reproducible, pero no compromete datos.
Estado	Abierto
Recomendación (no implementada)	Propuesta, no aplicada (no se modifica código de producción): validar el resultado de findPet() y lanzar una excepción asociada a 404
Comprobación posterior	Pendiente de corrección por el responsable del producto. S08 se reclasificó a fallido en la revisión del 30/10/2026 (reclasificación documental, sin nueva ejecución)
Evidencia	06_seguridad_MySQL/respuestas/S08_*.http.txt; registro del servidor

DEF-P02 · Defecto del producto (integridad bajo concurrencia)

Campo	Descripción
Caso / requisito	E01 / RF04 (RN02, RN04)
Reproducción	Enviar en paralelo 4 altas de la misma mascota para un propietario nuevo (equivale a un doble envío del formulario).
Esperado	Una mascota; las demás solicitudes rechazadas; ninguna mascota sin propietario
Obtenido	Más de una respuesta 302 por propietario. La segunda solicitud ejecuta 'update pets set owner_id=null ...' y desvincula la mascota confirmada. MySQL: 2 huérfanas (10 prop.) y 7 (20 prop.). H2: 18 huérfanas (20 prop.)
Impacto	Integridad: se pierde la asociación de una mascota confirmada al usuario; incumple RN02.
Severidad / prioridad	Alta / Media. Alta: afecta datos persistidos y el sistema informa éxito; requiere envíos simultáneos, plausibles con un doble clic.
Estado	Abierto
Recomendación (no implementada)	Propuesta: control de concurrencia optimista (@Version) en Owner o asociación gestionada desde Pet; como mitigación, impedir el doble envío en la interfaz
Comprobación posterior	Pendiente; repetir E01 tras la corrección
Evidencia	07_exploratorio_duplicado_MySQL, 07b_..._repeticion, 07c_..._H2, 07d_traza_sql_MySQL

DEF-P03 · Defecto del producto (manejo de errores, solo MySQL)

Campo	Descripción
Caso / requisito	E01 / RF04 (RN04)
Reproducción	Mismo escenario que DEF-P02 sobre MySQL. La violación de unicidad se reporta con la clave 'pets.owner_id'.
Esperado	Mensaje 'already exists' en el campo name, como ocurre con H2
Obtenido	HTTP 500 en 28 de 40 y 51 de 80 solicitudes. isDuplicatePetNameViolation() busca 'unique_owner_pet_name', nombre que solo existe en el esquema H2
Impacto	Disponibilidad parcial y mensajes: el duplicado concurrente termina en HTTP 500 en MySQL en lugar del mensaje de validación.
Severidad / prioridad	Media / Media. Media: no altera datos por sí mismo y ocurre solo bajo concurrencia en MySQL.
Estado	Abierto
Recomendación (no implementada)	Propuesta: nombrar la restricción en db/mysql/schema.sql o detectar la violación por código de error
Comprobación posterior	Pendiente
Evidencia	07_exploratorio_duplicado_MySQL, 07b_..._repeticion; registro del servidor

17. Métricas y análisis de resultados

17.1. Resultados de la última versión

Conjunto	Registrados	Aprobados	Fallidos	Bloqueados	Observación
Unitarias e integración	6	6	0	0	0
Funcionales HTTP	31	31	0	0	0
Seguridad básica	13	7	5	0	1
Adicional de concurrencia	1	0	1	0	0
Total	**51**	**44**	**6**	**0**	**1**

Sobre 50 casos evaluables (se excluye la observación S01), la tasa de aprobación es 44/50 = 88,0 %. Los 6 fallidos son S02, S08, S11, S12, S13, E01: la prueba adicional de concurrencia, el manejo no controlado del error en S08 y cuatro controles de configuración ausentes. Ninguno de los 37 casos planificados para RF01–RF06 en los niveles unitario, de integración y funcional (CP y F) falla en la última versión. Estas cifras cuentan casos; no se suman ejecuciones históricas, solicitudes de carga ni incidencias.

17.2. Ejecuciones históricas y revisiones

Conjunto	Ejecución o revisión	Casos	Aprobados	Fallidos	Observaciones
CP01–CP06	R0 transferencia y R1, R1b originales	6 por ejecución	4	2	INC-T01 (fixture)
CP01–CP06	R2 (H2) y R3 (MySQL) corregidos	6 por ejecución	6	0	-
F01–F31	Corrida 1 (29/10, script v1)	31	29	2	INC-T02 (script)
F01–F31	Corrida 2 (29/10, script v1)	31	31	0	Sin verificar el texto de los mensajes (INC-T03)
F01–F31	Corrida 3 (30/10, script v2)	31	31	0	Código, mensaje y persistencia
S01–S13	Ejecución única (29/10)	13	8	4	1 observación (S01)
S01–S13	Revisión documental (30/10)	13	7	5	S08: aprobado -> fallido, sin nueva ejecución
E01	MySQL (10 y 20 prop.) y H2 (20 prop.)	1	0	1	DEF-P02, DEF-P03

17.3. Incidencias por categoría

Categoría	Total	Abiertas	Cerradas o registradas
Defectos del producto (DEF)	3	3	0
Hallazgos de seguridad (SEC)	4	4	0
Observaciones (OBS)	2	1	1 registrada
Instrumento de prueba (INC-T)	3	0	3
Ambiente (AMB)	3	0	3

Total: 15 registros. Severidad de los defectos del producto: uno alta (DEF-P02) y dos media (DEF-P01, DEF-P03).

17.4. Tiempos

Medición	Valor	Fuente
Ejecución Maven de CP01–CP06 corregidos (H2 / MySQL)	27,330 s / 25,369 s	02 y 03, ejecucion.log
Suite completa del proyecto (H2)	36,896 s	04, ejecucion.log
Funcionales F01–F31, corrida 3	5 s	05c, resultados.json
Seguridad S01–S13	1 s	06, resultados.json
Carga: p95 con 10 y 30 VU	21 ms y 16 ms	08, consola.log

17.5. Interpretación

Las validaciones de RF01 a RF06 se comportan según las reglas definidas cuando las operaciones se ejecutan una a una, tanto en H2 como en MySQL, y muestran los mensajes previstos. Las fallas iniciales de CP04, CP06, F01 y F11 se debieron a los instrumentos de prueba; su análisis evitó registrar como defectos del producto problemas que no lo eran.

Los defectos confirmados se concentran en situaciones no cubiertas por los casos iniciales: solicitudes simultáneas sobre el mismo propietario y acceso a mascotas no asociadas. DEF-P02 es el más relevante, porque el sistema confirma una operación cuyo resultado no conserva. Los hallazgos de seguridad describen controles ausentes en una aplicación de demostración y deben valorarse según el entorno de despliegue.

Las métricas se basan en 51 casos y dos escenarios de carga locales. No permiten generalizar sobre funcionalidades no seleccionadas ni sobre el comportamiento en producción.

17.6. Revisión de coherencia de estados

En la revisión del 30/10/2026 se contrastó, caso por caso, el resultado esperado con el obtenido y con el estado registrado. La revisión no modificó ningún resultado esperado.

Caso	Esperado documentado	Obtenido	Decisión
S08	Rechazo con error controlado; la mascota de B no cambia; no se crean registros	HTTP 500 por NullPointerException; la mascota no cambia	Aprobado -> **Fallido**. La integridad se cumple, pero la excepción no controlada incumple el manejo controlado del error. Reclasificación documental, sin nueva ejecución. DEF-P01 sigue abierto
S07	Se rechaza la operación; no se registra la visita; sin traza interna	HTTP 500 por excepción explícita del controlador; sin visita ni traza	Se mantiene **Aprobado**: se cumplen los tres elementos. El código 500 se registra en OBS-02
S09	Error controlado sin traza de pila	HTTP 500; la aplicación detecta el id inexistente y lanza una excepción explícita; página genérica sin trazas	Se mantiene **Aprobado**: el error lo detecta la aplicación y no expone detalles internos. El código 500, en lugar de 404, se registra en OBS-02

Para que una nueva ejecución sea coherente con esta decisión, se ajustó el criterio automático de S08 en scripts/seguridad_http.py: ahora exige un rechazo 4xx y no acepta un 500. El script ajustado no se volvió a ejecutar; la evidencia de S08 sigue siendo la ejecución del 29/10/2026.

Si el docente interpreta «error controlado» en S09 como una respuesta 4xx, S09 pasaría a fallido y los totales serían 43 aprobados, 7 fallidos y 86,0 %. Esa interpretación no se aplicó porque el esperado documentado no especifica un código HTTP.

18. Conclusiones

Las conclusiones finales corresponden al informe de la semana 18. La interpretación parcial de este avance se presenta en la sección 17.5.

19. Referencias

Jakarta EE. (2022). Jakarta Persistence 3.1 specification. Eclipse Foundation. https://jakarta.ee/specifications/persistence/3.1/

Locust. (2026). Locust documentation. https://docs.locust.io/

Microsoft. (2026). Playwright for Python. https://playwright.dev/python/

Minhas, N. M., Petersen, K., Börstler, J., & Wnuk, K. (2020). Regression testing for large-scale embedded software development—Exploring the state of practice. Information and Software Technology, 120, 106254. https://doi.org/10.1016/j.infsof.2019.106254

Noël, R., Taramasco, C., & Márquez, G. (2022). Standards, processes, and tools used to evaluate the quality of health information systems: Systematic literature review. Journal of Medical Internet Research, 24(3), e26577. https://doi.org/10.2196/26577

OWASP Foundation. (2021). OWASP Top 10:2021. https://owasp.org/Top10/

Shin, Y., Choi, Y., & Lee, W. J. (2013). Integration testing through reusing representative unit test cases for high-confidence medical software. Computers in Biology and Medicine, 43(5), 434–443. https://doi.org/10.1016/j.compbiomed.2013.01.024

Spring. (2026). Spring Boot reference documentation: Endpoints. https://docs.spring.io/spring-boot/reference/actuator/endpoints.html

Spring Projects. (2026). spring-petclinic [Software]. GitHub. https://github.com/spring-projects/spring-petclinic

Tran, H. K. V., Unterkalmsteiner, M., Börstler, J., & Ali, N. bin. (2021). Assessing test artifact quality—A tertiary study. Information and Software Technology, 139, 106620. https://doi.org/10.1016/j.infsof.2021.106620

Universidad Tecnológica del Perú. (2026a). Pruebas funcionales: conceptos y enfoques, diseño de casos y automatización [Material de clase, semana 7]. Curso Pruebas de Software.

Universidad Tecnológica del Perú. (2026b). Automatización de pruebas funcionales, pruebas exploratorias y pruebas ad hoc [Material de clase, semana 8]. Curso Pruebas de Software.

Universidad Tecnológica del Perú. (2026c). Pruebas de rendimiento: carga, estrés y herramientas de simulación [Material de clase, semana 9]. Curso Pruebas de Software.

Universidad Tecnológica del Perú. (2026d). Validación REST y optimización del desempeño [Material de clase, semana 10]. Curso Pruebas de Software.

Universidad Tecnológica del Perú. (2026e). Pruebas de seguridad en aplicaciones [Material de clase, semana 11]. Curso Pruebas de Software.

20. Anexos

Anexo A. Organización de evidencias

Las carpetas se encuentran en el repositorio del equipo (https://github.com/Mooneer17/spring-petclinic-apf2) y en la carpeta de entrega.

Carpeta	Contenido
00_transferencia	Logs y reportes originales recibidos (Windows, H2). No modificados.
01_reproduccion_original / 01b_original_en_MySQL	Pruebas originales sin cambios en H2 y MySQL (R1, R1b).
02_correccion_H2 / 03_correccion_MySQL	CP01–CP06 corregidos y DIAG01 (R2, R3).
04_suite_completa_H2	Suite completa del proyecto.
05a / 05b_funcionales_MySQL	Corridas 1 y 2 de F01–F31 (script v1): resultados.json y respuestas HTTP.
05c_funcionales_MySQL_corrida3_v2	Corrida 3 de F01–F31 (script v2, 30/10/2026): resultados.json, consola y respuestas HTTP.
06_seguridad_MySQL	S01–S13: resultados.json y respuestas HTTP.
07, 07b, 07c, 07d	E01 en MySQL y H2; consultas de filas huérfanas; traza SQL.
08_rendimiento_MySQL	Calentamiento y cargas de 10 y 30 VU: consola, CSV, reporte HTML y memoria.
09_navegador_MySQL	Capturas N01–N08 y recorrido.json.
AMBIENTE.txt / CAMBIOS_PRUEBAS.diff	Versiones de sistema y herramientas; diff de los archivos de prueba.

Anexo B. Pasos de reproducción

Desde la carpeta proyecto/, con un JDK 17 o superior y acceso a Maven Central:

```
# Unitarias e integración en H2
./mvnw test -Dtest='AvanceUnitTests,AvanceIntegrationTests,DiagnosticoFixtureTests' -
Dsurefire.failIfNoSpecifiedTests=false
# Las mismas pruebas sobre MySQL local (usuario y base de db/mysql/user.sql)
./mvnw test -Dtest='Avance*,DiagnosticoFixtureTests' -
Dsurefire.failIfNoSpecifiedTests=false \
  -Ddatabase=mysql -Dspring.datasource.url=jdbc:mysql://localhost:3306/petclinic \
  -Dspring.datasource.username=petclinic -Dspring.datasource.password=petclinic -
Dspring.sql.init.mode=always
# Aplicación sobre MySQL y pruebas HTTP (desde scripts/)
./mvnw package -DskipTests && java -jar target/spring-petclinic-4.0.0-SNAPSHOT.jar --
spring.profiles.active=mysql
python3 funcionales_http.py --out salida/funcionales --bd MySQL
python3 seguridad_http.py --out salida/seguridad --bd MySQL
python3 exploratorio_duplicado_concurrente.py --out salida/e01 --bd MySQL --owners 20
locust -f locustfile.py --headless -H http://localhost:8080 -u 10 -r 2 -t 180s --csv
salida/carga10
python3 navegador_flujos.py --out salida/navegador
```

En Linux, scripts/ejecutar_apf2.sh automatiza estos pasos; en Windows, herramientas/ejecutar_apf2.ps1. La guía GUIA_REPRODUCCION.md detalla requisitos y salidas esperadas.

Anexo C. Versiones

Sistema evaluado: repositorio oficial spring-projects/spring-petclinic, commit 500158f732419217507c7656904b8e6aa1bcc0d6; Spring Boot 4.1.0; java.version 17. Repositorio del equipo con las pruebas, scripts y evidencias: https://github.com/Mooneer17/spring-petclinic-apf2. Detalle de herramientas en evidencias/AMBIENTE.txt.

Anexo D. Capturas del navegador automatizado

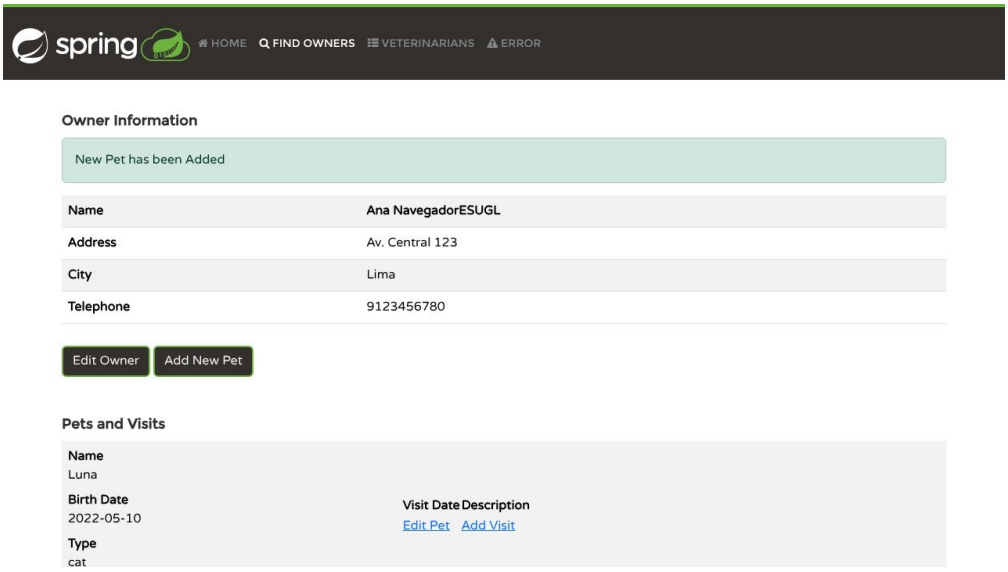


Figura 3. N05: mascota Luna registrada y asociada al propietario (RF04).

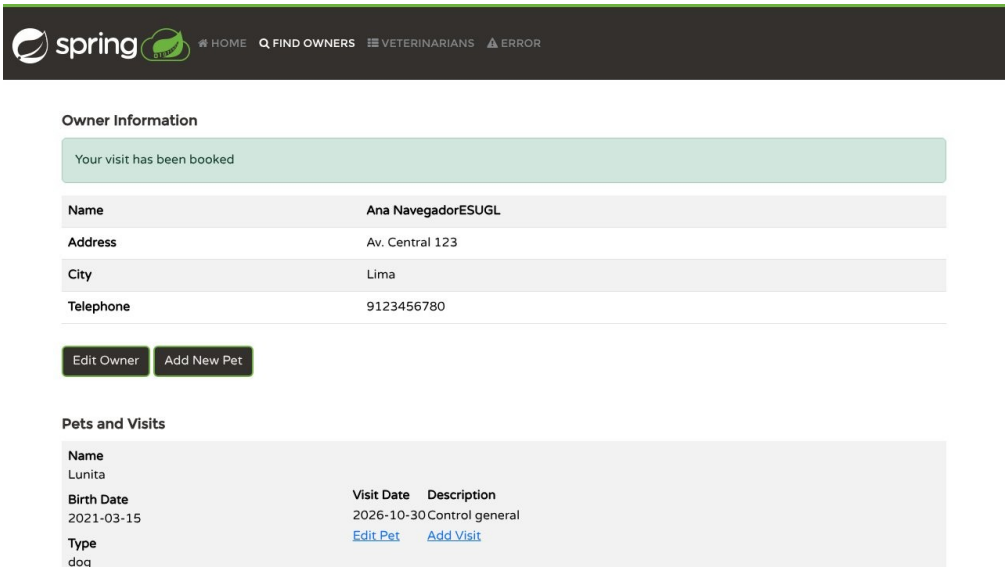


Figura 4. N08: visita del 30/10/2026 registrada para la mascota editada (RF06).

Anexo E. Corrección del helper pet() (INC-T01)

```
private Pet pet(Owner o) {
    Pet p = new Pet();
    p.setName("LunaAPF");
    p.setBirthDate(LocalDate.of(2022, 5, 10));
    p.setType(types.findPetTypes().iterator().next());
    o.addPet(p);
    owners.saveAndFlush(o);
    em.clear();                          // nuevo
    Pet persisted = owners.findById(o.getId()).orElseThrow()
            .getPet("LunaAPF", true);  // nuevo: mascota persistida
    assertThat(persisted).isNotNull();
    assertThat(persisted.getId()).isNotNull();
    return persisted;                    // antes: return p (id nulo)
}
```

Anexo F. Ficha complementaria de casos F, S y E01

Completa los campos que no figuran en las tablas de las secciones 11 y 13. Los pasos están documentados: corresponden al procedimiento que ejecutan los scripts conservados en el repositorio. Precondiciones comunes: para F01–F31, aplicación en ejecución con perfil mysql

y datos semilla, y cada caso crea sus registros con un sufijo único; para S01–S13, instancia local propia con peticiones anónimas; para E01, instancias locales con MySQL y H2 y propietarios nuevos sin mascotas.

ID	Objetivo	Técnica o control	Pasos documentados	Evidencia
F01	Verificar el registro de un propietario con datos válidos y su persistencia.	Escenario positivo (positivo)	1) Enviar POST /owners/new con los datos. 2) Seguir la redirección al detalle. 3) Verificar mensaje y datos mostrados.	05c_funcionales_MySQL_corrida3_v2/respuestas/F01_post.http.txt
F02	Verificar el rechazo del teléfono con un dígito menos que el exigido.	Valores límite (justo debajo: 9) (negativo)	1) Enviar POST /owners/new con el teléfono indicado. 2) Verificar el mensaje del campo. 3) Buscar el apellido usado para confirmar que no se guardó.	05c_funcionales_MySQL_corrida3_v2/respuestas/F02_post.http.txt
F03	Verificar el rechazo del teléfono con un dígito más que el exigido.	Valores límite (justo encima: 11) (negativo)	1) Enviar POST /owners/new con el teléfono indicado. 2) Verificar el mensaje del campo. 3) Buscar el apellido usado para confirmar que no se guardó.	05c_funcionales_MySQL_corrida3_v2/respuestas/F03_post.http.txt
F04	Verificar el rechazo de un teléfono no numérico.	Partición de equivalencia (clase no numérica) (negativo)	1) Enviar POST /owners/new con el teléfono indicado. 2) Verificar el mensaje del campo. 3) Buscar el apellido usado para confirmar que no se guardó.	05c_funcionales_MySQL_corrida3_v2/respuestas/F04_post.http.txt
F05	Verificar el rechazo de dirección vacía y ciudad en blanco.	Partición de equivalencia (vacío y en blanco) (negativo)	1) Enviar POST /owners/new con dirección vacía y ciudad con espacios. 2) Verificar ambos mensajes. 3) Buscar el apellido usado.	05c_funcionales_MySQL_corrida3_v2/respuestas/F05_post.http.txt
F06	Verificar el límite de longitud del apellido.	Valores límite (30 / 31) (positivo y negativo)	1) Enviar POST /owners/new con apellido de 30 caracteres. 2) Repetir con 31 caracteres. 3) Verificar respuesta y mensaje.	05c_funcionales_MySQL_corrida3_v2/respuestas/F06_30car.http.txt
F07	Verificar que una coincidencia única lleva al detalle del propietario.	Partición de equivalencia (coincidencia única) (positivo)	1) Crear un propietario con apellido único. 2) GET /owners?lastName=<apellido>. 3) Seguir la redirección y verificar el propietario mostrado.	05c_funcionales_MySQL_corrida3_v2/respuestas/F07_get.http.txt
F08	Verificar que varias coincidencias se presentan en una lista.	Partición de equivalencia (varias coincidencias) (positivo)	1) Crear dos propietarios con el mismo prefijo de apellido. 2) GET /owners?lastName=<prefijo>. 3) Verificar que la lista incluye ambos.	05c_funcionales_MySQL_corrida3_v2/respuestas/F08_get.http.txt
F09	Verificar el mensaje cuando no hay coincidencias.	Partición de equivalencia (sin coincidencias) (negativo)	1) GET /owners?lastName=<apellido inexistente>. 2) Verificar el mensaje mostrado.	05c_funcionales_MySQL_corrida3_v2/respuestas/F09_get.http.txt
F10	Verificar que se ignoran los espacios alrededor del apellido buscado.	Partición de equivalencia (entrada con espacios) (positivo)	1) Crear un propietario. 2) GET /owners?lastName= con espacios alrededor del apellido. 3) Verificar la redirección.	05c_funcionales_MySQL_corrida3_v2/respuestas/F10_get.http.txt
F11	Verificar que la búsqueda vacía lista todos los propietarios con paginación.	Partición de equivalencia (entrada vacía) (positivo)	1) GET /owners?lastName= (vacío). 2) Verificar tabla de propietarios y enlace a la página 2.	05c_funcionales_MySQL_corrida3_v2/respuestas/F11_get.http.txt
F12	Verificar la actualización válida de un propietario y su persistencia.	Escenario positivo (positivo)	1) Crear un propietario. 2) POST /owners/{id}/edit con los nuevos valores. 3) Seguir la redirección y verificar mensaje y datos.	05c_funcionales_MySQL_corrida3_v2/respuestas/F12_post.http.txt
F13	Verificar que la actualización conserva la validación del teléfono.	Partición de equivalencia (longitud inválida) (negativo)	1) Crear un propietario. 2) POST /owners/{id}/edit con teléfono 12345. 3) Verificar el mensaje. 4) Consultar el detalle.	05c_funcionales_MySQL_corrida3_v2/respuestas/F13_post.http.txt
F14	Verificar que la actualización conserva la obligatoriedad del nombre.	Partición de equivalencia (vacío) (negativo)	1) Crear un propietario. 2) POST /owners/{id}/edit con nombre vacío. 3) Verificar el mensaje. 4) Consultar el detalle.	05c_funcionales_MySQL_corrida3_v2/respuestas/F14_post.http.txt
F15	Verificar el registro de una mascota válida asociada al propietario.	Escenario positivo (positivo)	1) Crear un propietario. 2) POST /owners/{id}/pets/new con los datos. 3) Seguir la redirección y verificar mensaje y mascota asociada.	05c_funcionales_MySQL_corrida3_v2/respuestas/F15_post.http.txt
F16	Verificar el rechazo de una mascota sin nombre.	Partición de equivalencia (vacío) (negativo)	1) Crear un propietario. 2) POST /owners/{id}/pets/new con el dato indicado. 3) Verificar el mensaje. 4) Consultar el detalle del propietario.	05c_funcionales_MySQL_corrida3_v2/respuestas/F16_post.http.txt
F17	Verificar el rechazo de un nombre de mascota repetido para el mismo propietario.	Partición de equivalencia (nombre duplicado) (negativo)	1) Crear un propietario con la mascota Luna. 2) POST otra mascota con nombre 'LUNA'. 3) Verificar el mensaje. 4) Contar las mascotas con ese nombre.	05c_funcionales_MySQL_corrida3_v2/respuestas/F17_post.http.txt
F18	Verificar el rechazo de una fecha de nacimiento futura.	Valores límite (mañana) (negativo)	1) Crear un propietario. 2) POST /owners/{id}/pets/new con el dato indicado. 3) Verificar el mensaje. 4) Consultar el detalle del propietario.	05c_funcionales_MySQL_corrida3_v2/respuestas/F18_post.http.txt
F19	Verificar que la fecha de nacimiento igual a hoy se acepta.	Valores límite (hoy) (positivo)	1) Crear un propietario. 2) POST /owners/{id}/pets/new con fecha de hoy. 3) Seguir la redirección y verificar mensaje y mascota.	05c_funcionales_MySQL_corrida3_v2/respuestas/F19_post.http.txt
F20	Verificar el límite de longitud del nombre de la mascota.	Valores límite (30 / 31) (positivo y negativo)	1) Crear un propietario. 2) POST mascota con nombre de 30 caracteres. 3) Repetir con 31 caracteres. 4) Verificar respuesta y mensaje.	05c_funcionales_MySQL_corrida3_v2/respuestas/F20_30car.http.txt
F21	Verificar el rechazo de una mascota sin fecha de nacimiento.	Partición de equivalencia (ausente) (negativo)	1) Crear un propietario. 2) POST /owners/{id}/pets/new con el dato indicado. 3) Verificar el mensaje. 4) Consultar el detalle del propietario.	05c_funcionales_MySQL_corrida3_v2/respuestas/F21_post.http.txt
F22	Verificar la edición de nombre, tipo y fecha de una mascota y su persistencia.	Escenario positivo (positivo)	1) Crear propietario y mascota Luna. 2) POST /owners/{o}/pets/{p}/edit con los nuevos datos. 3) Seguir la redirección; verificar mensaje, id y datos.	05c_funcionales_MySQL_corrida3_v2/respuestas/F22_post.http.txt
F23	Verificar que la edición rechaza un nombre ya usado por otra mascota del propietario.	Partición de equivalencia (nombre duplicado) (negativo)	1) Crear propietario con Luna y Max. 2) POST edición de Max con nombre Luna. 3) Verificar el mensaje. 4) Consultar el detalle.	05c_funcionales_MySQL_corrida3_v2/respuestas/F23_post.http.txt

ID	Objetivo	Técnica o control	Pasos documentados	Evidencia
F24	Verificar que la edición rechaza una fecha de nacimiento futura.	Valores límite (mañana) (negativo)	1) Crear propietario y mascota. 2) POST edición con fecha de mañana. 3) Verificar el mensaje. 4) Consultar el detalle.	05c_funcionales_MySQL_corrida3_v2/respuestas/F24_post.http.txt
F25	Verificar que la edición rechaza un nombre vacío y conserva el dato anterior.	Partición de equivalencia (vacío) (negativo)	1) Crear propietario y mascota. 2) POST edición con nombre vacío. 3) Verificar el mensaje. 4) Consultar el detalle.	05c_funcionales_MySQL_corrida3_v2/respuestas/F25_post.http.txt
F26	Verificar el registro de una visita válida y su asociación con la mascota.	Escenario positivo; valores límite (mañana) (positivo)	1) Crear propietario y mascota. 2) POST /owners/{o}/pets/{p}/visits/new. 3) Seguir la redirección; verificar mensaje y visita.	05c_funcionales_MySQL_corrida3_v2/respuestas/F26_post.http.txt
F27	Verificar el rechazo de una visita con fecha de hoy.	Valores límite (hoy) (negativo)	1) Crear propietario y mascota. 2) POST /owners/{o}/pets/{p}/visits/new con el dato indicado. 3) Verificar el mensaje. 4) Consultar el detalle.	05c_funcionales_MySQL_corrida3_v2/respuestas/F27_post.http.txt
F28	Verificar el rechazo de una visita con fecha pasada.	Partición de equivalencia (fechas pasadas) (negativo)	1) Crear propietario y mascota. 2) POST /owners/{o}/pets/{p}/visits/new con el dato indicado. 3) Verificar el mensaje. 4) Consultar el detalle.	05c_funcionales_MySQL_corrida3_v2/respuestas/F28_post.http.txt
F29	Verificar el rechazo de una visita sin descripción.	Partición de equivalencia (vacío) (negativo)	1) Crear propietario y mascota. 2) POST /owners/{o}/pets/{p}/visits/new con el dato indicado. 3) Verificar el mensaje. 4) Consultar el detalle.	05c_funcionales_MySQL_corrida3_v2/respuestas/F29_post.http.txt
F30	Verificar el rechazo de una descripción compuesta solo por espacios.	Partición de equivalencia (en blanco) (negativo)	1) Crear propietario y mascota. 2) POST /owners/{o}/pets/{p}/visits/new con el dato indicado. 3) Verificar el mensaje. 4) Consultar el detalle.	05c_funcionales_MySQL_corrida3_v2/respuestas/F30_post.http.txt
F31	Verificar el límite de longitud de la descripción de la visita.	Valores límite (255 / 256) (positivo y negativo)	1) Crear propietario y mascota. 2) POST visita con descripción de 255 caracteres. 3) Repetir con 256 caracteres. 4) Verificar respuestas y mensajes.	05c_funcionales_MySQL_corrida3_v2/respuestas/F31_255car.http.txt
S01	Determinar si la variante evaluada exige autenticación para consultar o modificar datos.	Autenticación; caso negativo	1) Enviar GET anónimo a /, /owners/find, /owners?lastName=, /owners/1, /owners/1/edit, /vets.html y /owners/new. 2) Enviar GET /login. 3) Registrar los códigos HTTP.	06_seguridad_MySQL/respuestas/S01_owners_new.http.txt
S02	Comprobar si los endpoints de administración están expuestos sin autenticación.	Configuración de seguridad; caso negativo	1) GET anónimo a /actuator, /actuator/env, /configprops, /beans y /mappings. 2) HEAD /actuator/heapdump. 3) Registrar códigos y revisar si /env enmascara valores.	06_seguridad_MySQL/respuestas/S02_actuator.http.txt
S03	Comprobar que el contenido almacenado se presenta escapado y no se ejecuta como script.	Validación de entradas; caso negativo	1) Registrar un propietario con address='<script>alert(1)</script>' y city='Lima'. 2) Abrir su detalle. 3) Buscar la etiqueta sin escapar y la versión escapada en el HTML.	06_seguridad_MySQL/respuestas/S03_detalle.http.txt
S04	Comprobar que la búsqueda trata cargas de inyección SQL como texto.	Validación de entradas; caso negativo	1) GET /owners?lastName= con tres cargas de inyección SQL. 2) Verificar respuesta, mensaje y ausencia de trazas. 3) Consultar un apellido semilla para confirmar que la tabla sigue operativa.	06_seguridad_MySQL/respuestas/S04_p0.http.txt
S05	Comprobar que el formulario de alta no permite fijar el identificador del propietario.	Validación de entradas (parámetros no permitidos); caso negativo	1) POST /owners/new con un parámetro adicional id=1. 2) Obtener el id asignado. 3) Consultar /owners/1 y comparar con su estado previo.	06_seguridad_MySQL/respuestas/S05_post.http.txt
S06	Comprobar que el formulario de edición no permite modificar otro propietario mediante el id.	Validación de entradas (parámetros no permitidos); caso negativo	1) Crear propietarios A y B. 2) POST /owners/A/edit con id=B y ciudad Cusco. 3) Consultar A y B.	06_seguridad_MySQL/respuestas/S06_post.http.txt
S07	Comprobar que no se registra una visita para una mascota que no pertenece al propietario de la URL.	Integridad propietario–mascota; caso negativo	1) Crear propietarios A y B, y una mascota de B. 2) GET y POST /owners/A/pets/{mascotaB}/visits/new. 3) Consultar A y B para confirmar que no se registró la visita.	06_seguridad_MySQL/respuestas/S07_post.http.txt
S08	Comprobar que no se edita una mascota de otro propietario y que el rechazo es controlado.	Integridad propietario–mascota; caso negativo	1) Crear propietarios A y B, y la mascota Bruno de B. 2) GET y POST /owners/A/pets/{mascotaB}/edit con nombre 'Hackeado'. 3) Consultar A y B.	06_seguridad_MySQL/respuestas/S08_post.http.txt
S09	Comprobar que un propietario inexistente produce un error controlado sin divulgar trazas.	Configuración de seguridad (manejo de errores); caso negativo	1) GET /owners/999999. 2) Revisar código HTTP, mensaje mostrado y presencia de trazas.	06_seguridad_MySQL/respuestas/S09_get.http.txt
S10	Comprobar el rechazo de entradas sobredimensionadas sin errores del servidor.	Validación de entradas; caso negativo	1) POST /owners/new con dirección de 256 y 20000 caracteres y ciudad de 81. 2) Verificar respuesta y mensaje de validación.	06_seguridad_MySQL/respuestas/S10_address256.http.txt
S11	Comprobar si los formularios rechazan solicitudes sin token CSRF.	Configuración de seguridad (CSRF); caso negativo	1) Revisar si el formulario /owners/new incluye un token CSRF. 2) POST /owners/new con cabeceras Origin y Referer de otro sitio y sin token. 3) Registrar la respuesta.	06_seguridad_MySQL/respuestas/S11_post_origen_externo.http.txt
S12	Comprobar la presencia de cabeceras de seguridad HTTP básicas.	Configuración de seguridad (cabeceras); caso negativo	1) GET /. 2) Revisar X-Content-Type-Options, X-Frame-Options, Content-Security-Policy y Cache-Control.	06_seguridad_MySQL/respuestas/S12_get.http.txt
S13	Comprobar que el identificador de sesión no se expone en la URL.	Configuración de seguridad (sesión); caso negativo	1) Desde un cliente sin cookie, POST /owners/new. 2) Revisar si Location contiene ';jsessionid='.	06_seguridad_MySQL/respuestas/S13_post_sin_cookie.http.txt
E01	Comprobar la integridad de RN02 y RN04 ante altas simultáneas de la misma mascota.	Prueba adicional de concurrencia (envíos simultáneos) (negativo)	Crear propietario; enviar 4 POST /owners/{id}/pets/new en paralelo; contar mascotas visibles y filas huérfanas	07*_exploratorio_*, evidencias/07d_traza_sql_MySQL/traza_sql_pets.txt